Assignment: 2

CS3318

2026-2027

Team Software Modernisation

Assignment Due: November 22nd 2026

1 Objective

In this assignment you will work as part of a team to modernise Java-RayTracer, a Java ray-tracing application developed by TetroVolt, which had its last update about 8 years ago. The repository provides a working (in JDK 8) ray tracer but exhibits characteristics commonly found in legacy software systems, including limited documentation, unclear architectural boundaries, lack of an API layer, and probable technical debt.

The goal of this assignment is to apply software maintenance, refactoring, and modernisation techniques to transform the codebase into a maintainable, reusable software product suitable for further development.

The original code repository can be found at

- https://github.com/TetroVolt/Java-RayTracer

You should not clone this repository as your team must use a local shared repository on `csgate.ucc.ie` that will be used for grading.

1.1 Skills Tested

- Work within a software development team using collaborative tools.
- Identify and evaluate technical debt in an existing codebase.
- Modernise an existing codebase.
- Design and implement an API that captures core functionality.
- Produce software documentation using Javadoc.
- Implement an example application using the API.
- Use team software development tools, such as Maven and Git.

2 Assignment Requirements

2.1 Technical Debt

Conduct an assessment of the current codebase and identify examples of technical debt. Technical debt includes

- Poor class cohesion and excessive coupling.

- Code duplication.
- Sparse or missing documentation.
- Missing test or untestable code.
- Naming inconsistencies and naming not adhering to language conventions.

It is useful to create a Technical Debt Register that lists the technical debt to plan the software modernisation task. An example of a technical debt registry is

ID	Description & Component	Category	Impact	Effort	Priority
TD-101	Inconsistent naming in class Books	Code Style	Low	Low	Low

2.2 Architecture Modernisation

Java-RayTracer is written as a standalone application. Your team must redesign the architecture to separate the ray-tracing engine from demonstration code. Create a module to capture the ray tracing engine and present its functionality as an API. The API should feature a public interface that could be reused by other applications.

The ray tracing API must be used to create an example application. The example application should depend upon the API, but the API must not depend on the example application.

2.3 Update the Codebase

Separating the codebase into the core functionality and an example application will give opportunities to improve the maintainability of the code. As part of this process your team should look for opportunities to

- Use modern Java language features.
- Improve encapsulation.
- Include or improve package and module organisation.
- Use appropriate design patterns.
- Identify opportunities for program correctness e.g. design by contract.
- Include or improve exception handling.

All code modifications should be justified.

2.4 Documentation

Developer level documentation must be added to the project. Your team should document the API by including Javadoc for all visible API packages, classes, interfaces and methods.

2.5 Repository Management

Your team will need to manage the team's shared repository. There is no requirement to use a managed service. However, your team should follow some best practices, such as

- Keeping a protected main branch.
- Using feature branch workflows.
- Meaningful commit messages.

The Git repository should demonstrate active participation and will be used in the grading.

3 Deliverables

- Updated project in the group's repository, containing a well documented API and an example application.
- A short written report describing the choices your team made about the API, the management of the project and any issues your team encountered.